



UNIVERSIDAD MODELO

Escuela de Ingeniería

Desarrollo de Tecnología y de Software (DTS)

Algoritmos

Profesor: Edson G. Estrada López

Proyecto #4 Organizador de horarios escolares

Aguilar de la Garza Andrés, Martínez Ortega Gustavo Luis, López Trujillo Santiago y Salgado López Leonardo.

27/04/2026



Objetivos

Objetivo General

Desarrollar un algoritmo basado en backtracking que genere un horario válido para una licenciatura, cumpliendo un conjunto de restricciones académicas, de recursos y de disponibilidad.

Objetivo Específico

- Analizar la problemática para garantizar el entendimiento del problema y la manera de solución adecuada, utilizando conceptos de backtracking.
- Demostrar los conocimientos adquiridos desarrollando exitosamente un algoritmo en python que genere un horario con todos los requerimientos especificados.



Descripción general del proyecto



El proyecto consiste en realizar un organizador de horarios escolares, con la finalidad de tomar un gran conjunto de profesores y materias con sus respectivas restricciones para poder realizar un horario escolar para los grupos de 4 años escolares, e igualmente asignarlos a su salón correspondiente.

El algoritmo va buscando todas las posibilidades de calendario hasta encontrar una que cumpla de manera correcta con todos los requerimientos especificados previamente.



Tabla de requerimientos

1.El algoritmo construye un horario semanal para 4 semestres (2°, 4°, 6°, 8°), asignando: Días, Bloques de horario, asignaturas, profesores y salones.	
2.El algoritmo deberá explorar todas las combinaciones posibles hasta encontrar una válida que cumpla con todas las restricciones.	
3.La estructura de horario marca 5 días disponibles (lunes a viernes) y bloques por día, siendo: a.Bloque A. 07:00 – 09:00 hrs. b.Bloque B: 09:00 – 11:00 hrs. c.Bloque C: 11:00 – 13:00 hrs.	
4.En la estructura académica, a.Los semestres considerados son 2°, 4°, 6° y 8° b.Cada semestre tiene 6 asignaturas c.Cada semestre tiene un salón fijo asignado.	
5.Respecto a las asignaturas, cada una deberá especificar: a.El número de bloques semanales requeridos, 1, 2 o 3 b.El profesor asignado el tipo de espacio requerido: i.Salón convencional ii.Salón de cómputo.	

Tabla de requerimientos

6. Para los profesores: a. Existe un conjunto de N profesores b. Cada profesor puede impartir máximo 1 asignatura por semestre c. Puede no tener asignaturas asignadas d. Tiene restricciones de disponibilidad por: i. Día ii. Bloque.	
7. Para los recursos físicos: a. Cada semestre tiene un salón propio (sin conflictos entre semestres) b. Existen 2 salones de cómputo: i. Laboratorio A ii. Laboratorio B c. Restricciones: i. Un laboratorio no puede ser usado por más de una asignatura en el mismo bloque. ii. Las asignaturas que requieren laboratorio deben asignarse obligatoriamente a uno disponible.	



Tabla de requerimientos

8.El algoritmo debe garantizar: - a.No traslapes por semestre: Un semestre no puede tener más de una asignatura en el mismo bloque. - b.Cumplimiento de bloques por asignatura: Cada asignatura debe cubrir exactamente su número requerido de bloques. - c.No se repetirá una asignatura el mismo día: Una asignatura no puede ser programada más de un bloque dentro del mismo día. - d.Asignación válida de profesores: Un profesor: - i.No puede ser asignado a más de una clase en el mismo bloque. - ii.No puede exceder 1 asignatura por semestre - iii.Debe respetar su disponibilidad	
9.Uso de laboratorios: - i.Máximo 2 asignaturas simultáneas que requieran cómputo - i.Solo asignaturas que lo que requieran pueden ocuparlos	
Huecos en el horario (opcional): Se permite que un semestre tenga bloques sin asignatura (configurable)	



Marco teórico

¿Qué es el Backtracking?

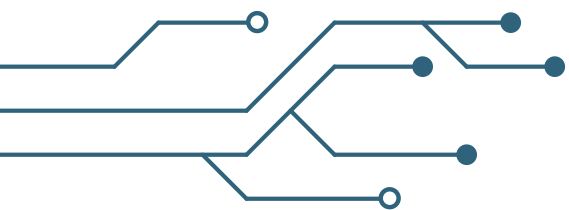
El *backtracking* es una técnica algorítmica para la solución de diversas problemáticas, esta consiste en hallar una solución de forma que prueba diferentes opciones y deshaciendo estas si conducen a un “callejón sin salida”, una ruta o solución que no permite seguir avanzando hasta la solución deseada por encontrar. Cuando se llega a este punto sin salida, el algoritmo retroceda hasta un punto o nodo de decisión anterior, y explora un camino diferente hasta encontrar una solución o agotar todas las posibilidades. Estos algoritmos buscan de manera recursiva todas las posibles soluciones a un problema.

¿Qué es un diccionario?

Los diccionarios, son una forma de almacenar elementos de manera similar a las listas, pero en lugar de acceder a los elementos utilizando su índice, se les asigna una clave fija y se accede al elemento utilizando la clave, lo que se maneja es un par tipo *key-value*, que es una estructura de datos mucho más adecuada para ciertos problemas. Son colecciones de elementos a los que se accede por una clave concreta. Los elementos de un diccionario pueden tener cualquier tipo de datos. Igualmente existe la comprensión de diccionarios, que permite transformar un diccionario en otro diccionario, siendo útil para sustituir bucles for y funciones lambda.

Diccionario con matrices

Como los diccionarios pueden almacenar distintos tipos de datos, dentro de esta ejecución se están empleando de manera muy similar a una clase, para poder asignar distintos valores a variables de cada uno de los “objetos” empleados en la resolución del problema.



Manejo de Diccionarios

En este algoritmo manejamos varios diccionarios, tenemos uno de materias, uno de los salones de computo, uno de los profesores y uno con todos los horarios de cada semestre.

Cada diccionario tiene información importante, que posteriormente será usada en la siguiente función, esto nos permite tener mayor accesibilidad a la información.

```
materias = [
    # Semestre 0
    {"materia": "Calculo", "bloques": 3, "computo": True, "semestre": 0, "profesor": profesores[1], "salon": "151"},
    {"materia": "Algebra", "bloques": 3, "computo": False, "semestre": 0, "profesor": profesores[0], "salon": "151"},
    {"materia": "Redes", "bloques": 2, "computo": True, "semestre": 0, "profesor": profesores[2], "salon": "151"},
    {"materia": "Programacion", "bloques": 3, "computo": True, "semestre": 0, "profesor": profesores[3], "salon": "151"},
    {"materia": "Ingles", "bloques": 2, "computo": False, "semestre": 0, "profesor": profesores[4], "salon": "151"},
    {"materia": "Etica", "bloques": 1, "computo": False, "semestre": 0, "profesor": profesores[5], "salon": "151"},

    # Semestre 1
    {"materia": "Algoritmos", "bloques": 3, "computo": False, "semestre": 1, "profesor": profesores[6], "salon": "160"},
    {"materia": "Proyectos", "bloques": 3, "computo": False, "semestre": 1, "profesor": profesores[7], "salon": "160"},
    {"materia": "Fisica", "bloques": 2, "computo": False, "semestre": 1, "profesor": profesores[8], "salon": "160"},
    {"materia": "BasesDatos", "bloques": 3, "computo": True, "semestre": 1, "profesor": profesores[9], "salon": "160"},
    {"materia": "Circuitos", "bloques": 2, "computo": False, "semestre": 1, "profesor": profesores[0], "salon": "160"},
    {"materia": "Probabilidad", "bloques": 1, "computo": False, "semestre": 1, "profesor": profesores[1], "salon": "160"},

    # Semestre 2
    {"materia": "Sistemas", "bloques": 3, "computo": False, "semestre": 2, "profesor": profesores[2], "salon": "200"},
    {"materia": "Algebraa", "bloques": 3, "computo": False, "semestre": 2, "profesor": profesores[3], "salon": "200"},
    {"materia": "Calculoa", "bloques": 2, "computo": False, "semestre": 2, "profesor": profesores[4], "salon": "200"},
    {"materia": "Arquitectura", "bloques": 3, "computo": True, "semestre": 2, "profesor": profesores[5], "salon": "200"},
    {"materia": "Electronica", "bloques": 2, "computo": False, "semestre": 2, "profesor": profesores[6], "salon": "200"},
    {"materia": "Estadistica", "bloques": 1, "computo": False, "semestre": 2, "profesor": profesores[7], "salon": "200"},

    # Semestre 3
    {"materia": "Redesa", "bloques": 3, "computo": False, "semestre": 3, "profesor": profesores[8], "salon": "210"},
    {"materia": "Algoritmosa", "bloques": 3, "computo": False, "semestre": 3, "profesor": profesores[9], "salon": "210"},
    {"materia": "Proyectosa", "bloques": 2, "computo": False, "semestre": 3, "profesor": profesores[0], "salon": "210"},
    {"materia": "IA", "bloques": 3, "computo": True, "semestre": 3, "profesor": profesores[1], "salon": "210"},
    {"materia": "Seguridad", "bloques": 2, "computo": False, "semestre": 3, "profesor": profesores[2], "salon": "210"},
    {"materia": "Compiladores", "bloques": 1, "computo": True, "semestre": 3, "profesor": profesores[3], "salon": "210"}
]

computo = [
    {"horarios": np.full((3,5), True)},
    {"horarios": np.full((3,5), True)}
]
```



Creación de calendarios

En esta función se crea los calendarios vacíos, para posteriormente ser rellenos, de igual modo llama a la función principal "horario_impl".

En dado caso que la función falle, va a usar un print para decir que no es posible realizar este calendario. En dado caso que funcione sin ningún problema, mostrará todos los calendarios, incluyendo los de computo.

```
def Horario(materias, computo):
    calendarios = [
        {"semestre": "2", "calendario": np.full((3,5), "empty".ljust(30))},
        {"semestre": "4", "calendario": np.full((3,5), "empty".ljust(30))},
        {"semestre": "6", "calendario": np.full((3,5), "empty".ljust(30))},
        {"semestre": "8", "calendario": np.full((3,5), "empty".ljust(30))}
    ]

    pos = 0
    sandwich = True

    if not Horario_impl(pos, calendarios, materias, computo, sandwich):
        print("no es posible este calendario")
    else:
        # Si todo está correcto, imprimir
        for i in range(4):
            print(f'\n{calendarios[i]["calendario"]}\n')
        for i in range(2):
            print(f'\n{computo[i]["horarios"]}\n')

    return True
```

Descripción de la implementación

La primera parte de implementación es la creación de variables locales que se utilizaran en esa iteración, después se utilizan dos situaciones de parada:

- La primera que se usa cuando el código es correcto, comprueba que se hayan utilizado todos los módulos para decir que es correcta la solución.
- La segunda situación de parada es para comprobar que queden suficientes espacios libres para colocar las materias faltantes.

```
def Horario_impl(pos, calendarios, materias, computo, sandwich):  
    #variables  
    n_materias = len(materias)  
  
    semestre = pos // 15  
    localpos = pos % 15  
  
    col = localpos % 5  
    row = localpos // 5  
  
    #detenerse al llegar a la ultima poscion  
    if pos == 60:  
        cont=0  
        for i in range(n_materias):  
            if materias[i]["bloques"]==0:  
                cont+=1  
        if cont== n_materias:  
            return True  
        else:  
            return False  
  
    #checa espacios libres suficientes  
    espacios_validos = 0  
    modulos = 0  
    for m in range(n_materias):  
        if materias[m]["semestre"] == semestre:  
            if materias[m]["bloques"] != 0:  
                modulos += materias[m]["bloques"]  
  
    for x in range(3):  
        for y in range(5):  
            if calendarios[semestre]["calendario"][x][y] == "empty".ljust(30):  
                espacios_validos += 1  
  
    if modulos > espacios_validos:  
        return False
```

Descripción de la implementación

La función utiliza un for que recorre el número de materias para poder llamar correctamente el diccionario, comenzamos con comprobaciones si estamos buscando las del semestre correcto o si faltan por utilizar, en el caso de que no se saltan con un continue.

Después hacen las comprobaciones on road comenzando con comprobar la disponibilidad del maestro llamando a un diccionario con un arreglo de 3*5 booleano, después se comprueba que no esté puesta antes en esa columna la misma materia.

Por último, checa si esta un libre el salón de cómputo en caso de ser usado. Si pasa todo esto pone la materia en el calendario, marca el maestro como ocupado en esa hora y resta uno a la disponibilidad de esa clase.

```
#funcion principal
for num in range(n_materias):
    #inicializar
    invalido = False
    indexcomp = None

    #si la materia es de otro semestre o ya no le quedan clases que usar la salta
    if materias[num]["semestre"] != semestre or materias[num]["bloques"] == 0:
        continue

    #si en esta hora no se puede usar al maestro saltar
    if materias[num]["profesor"]["horarios"][row][col] == False:
        continue

    #si ya se usa la materia en el dia saltar
    for i in range(3):
        if materias[num]["materia"] == calendarios[semestre]["calendario"][i][col]:
            invalido = True
            break
    if invalido:
        continue

    #si usa el salon de computo checar si esta disponible
    if materias[num]["computo"] == True:
        for z in range(2):
            if computo[z]["horarios"][row][col] == True:
                indexcomp=z
                break
        if indexcomp is None:
            continue

    #rellenamos el horario con la materia disponible
    calendarios[semestre]["calendario"][row][col] = materias[num]["materia"]
    #marcamos esta hora del profe como ocupada
    materias[num]["profesor"]["horarios"][row][col] = False
    #restamos un modulo de los disponibles de la clase
    materias[num]["bloques"] -= 1
```



Descripción de la implementación

Comienza el backtracking llamandose a si mismo con pos+1, si regresa true, comienza a volver en cascada, si no hace lo siguiente

regresa los valores a los que tenía originalmente, y volverá a intentarlo con otra materia

Si se acaban las materias sale del for, pone la hora como "vacío" y comienza a buscar si hay huecos en el horario si no son permitidos, contando los vacíos con valores dependiendo de su posición y contando la cantidad de casillas no checadas,

se llama de a la función con pos+1 para seguir o en caso de regresar como false empezar la cascada de false

```
if indexcomp is not None:
    computo[indexcomp]["horarios"][row][col] = False

if Horario_impl(pos+1, calendarios, materias, computo, sandwich):
    return True

#backtracking
materias[num]["bloques"] += 1
calendarios[semestre]["calendario"][row][col] = "empty".ljust(30)
materias[num]["profesor"]["horarios"][row][col] = True

if indexcomp is not None:
    computo[indexcomp]["horarios"][row][col] = True

#intentamos poner una hora vacia, si no se puede por el sandwich returnea falso
calendarios[semestre]["calendario"][row][col] = "vacío"

cont=0
conte= 0
#comprobar sandiwches
if(sandwich == False):
    for i in range(3):
        if calendarios[semestre]["calendario"][i][col] == "vacío":
            cont += 1+i
        if calendarios[semestre]["calendario"][i][col] == "empty".ljust(30):
            conte += 1+i
    if cont == 2 and conte != 0:
        return False

#seguir despues de poner vacio
if Horario_impl(pos+1, calendarios, materias, computo, sandwich):
    return True

calendarios[semestre]["calendario"][row][col] == "empty".ljust(30)
return False
```



Semestre 6			
Martes	Miercoles	Jueves	Viernes
Sistemas	Sistemas	Algebraa	Algebraa
Calculoa	Calculoa	Electronica	Electronica
Arquitectura	Arquitectura	vacío	Arquitectura
Semestre 8			
Martes	Miercoles	Jueves	Viernes
Redesa	Algoritmosa	Redesa	IA
Proyectosa	Proyectosa	Seguridad	Seguridad
Algoritmosa	IA	Compiladores	vacío
Computo 1			
Martes	Miercoles	Jueves	Viernes
Vacío	Vacío	Vacío	Vacío
Vacío	Vacío	Ocuapdo	Ocuapdo
Ocuapdo	Ocuapdo	Ocuapdo	Ocuapdo
Computo 2			
Martes	Miercoles	Jueves	Viernes
Vacío	Vacío	Vacío	Vacío
Vacío	Vacío	Ocuapdo	Ocuapdo
Vacío	Vacío	Vacío	Vacío

Prueba y demostración

Al finalizar, el algoritmo generó de manera correcta los horarios de acuerdo a las especificaciones enlistadas de materias, profesores y bloques necesarios por materia, al igual que las materias que requirieron del salón de cómputo, para poder generar igualmente este horario.

Se puede concluir que las pruebas realizadas fueron exitosas, ya que igualmente pasaron todos los requerimientos planteados dentro de la lista de requerimientos y restricciones dentro del PDF del proyecto.



Listado de bugs conocidos



Dentro del código no aparenta haber bug alguno, sin embargo la versión de implementación está programada para solamente funcionar de acuerdo con los requerimientos de la asignación, por lo que si se llegasen a incluir más semestres o mayor cantidad de bloques (por ejemplo un turno vespertino para otros semestres) no se ejecutaría de forma correcta y requeriría cambios en la planificación del código.



Conclusión general

En conclusión, este proyecto proporcionó al equipo un reto el cual superar para poder aprender y demostrar los conocimientos de la materia al aplicar de forma dinámica una solución con backtracking.

Luego de la implementación del código para la solución de este caso por medio de las técnicas del backtracking y la programación en python, este método resultó ser una solución viable para un problema de alta complejidad combinatoria, debido a que las restricciones que tiene este caso de los horarios van acortando las posibilidades de solución de esta práctica.

El sistema logra respetar simultáneamente múltiples tipos de restricciones, como la disponibilidad de los profesores, el uso de las aulas de cómputo, la distribución de las materias y que estas cumplieren sus bloques necesarios demuestran como el backtracking escala bien para este tipo de problemas donde se requieren explorar todas las posibles soluciones.



Conclusiones Personales



Gustavo Luis Martínez Ortega

Durante el proceso de elaboración del proyecto nos enfrentamos probablemente al mayor desafío que habíamos enfrentando en tema de backtracking. Hubo muchos problemas sobretodo con el tiempo de ejecución, entonces fue muy satisfactorio poder ir resolviendo cada uno de esos problemas uno a uno hasta que el código logre funcionar verdaderamente bien.

Andrés Aguilar de la Garza

En el desarrollo de este código se probaron muchas más opciones que nunca, y a que fue el primero en no ser un algoritmo ya definido y con tantas formas de realizarse, se probaron muchos enfoques distintos se optó por uno con comprobaciones en la marcha y un backtraking simple, pues al comienzo se dependía mucho del backtraking y tardaba mucho en determinar que era imposible por lo que las comprobaciones recortaron significativamente estas búsquedas extra.



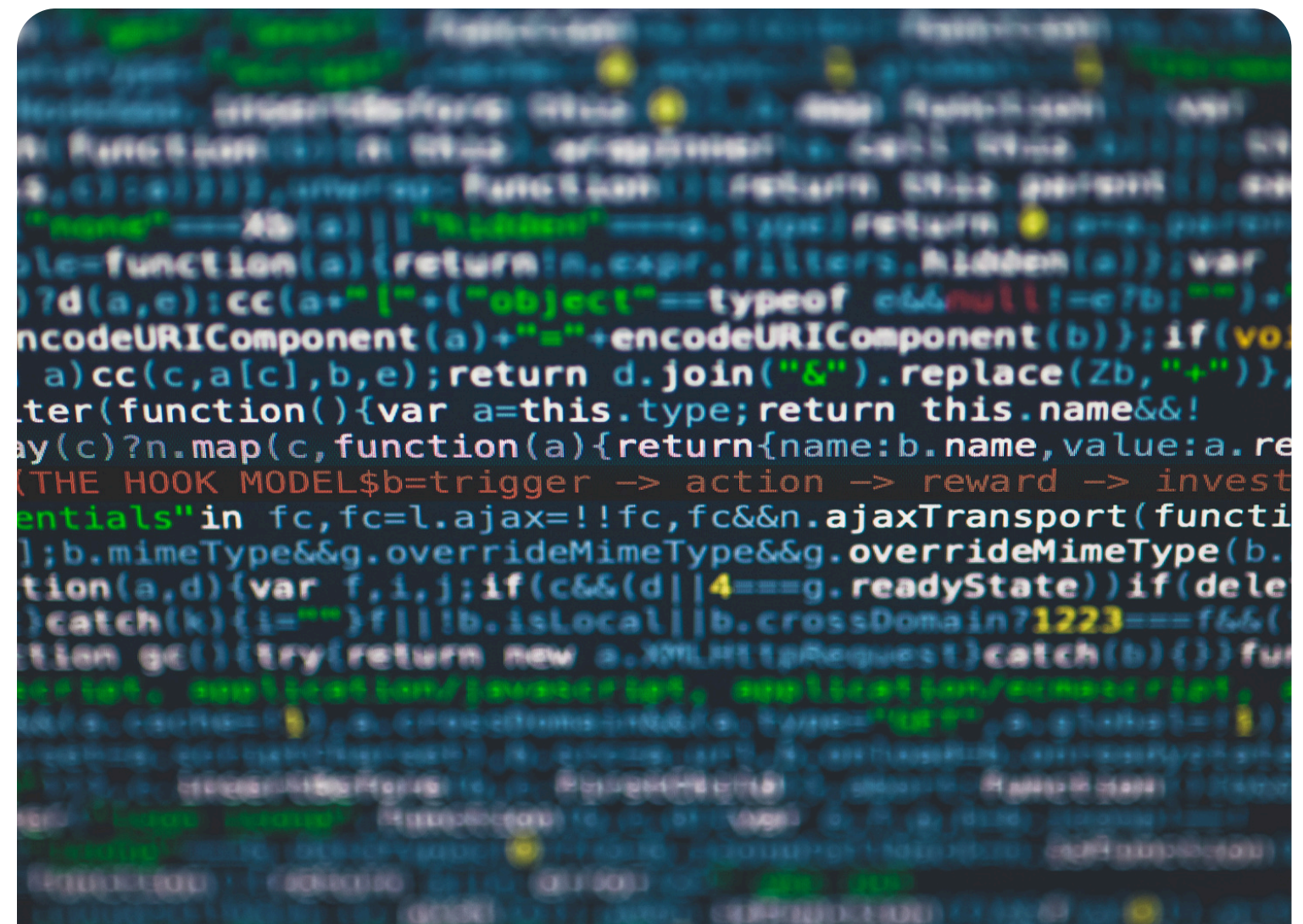
Conclusiones Personales

Leonardo Salgado López

Este proyecto fue bastante desafiante al tener que incorporar una infinidad de restricciones y comprobaciones para continuar la búsqueda de soluciones a la problemática, realmente muestra todo el trasfondo que una institución escolar debe conllevar y todas las variables a considerar para poder armar un horario escolar, algo que cualquiera puede pensar que es una tarea simple de realizar, cuando realmente no lo es, por la infinidad de posibles soluciones que uno manualmente tardaría en probar. Los logros que más me gustan son poder entender el proceso de comprobaciones previo al backtracking como el proceso de backtracking para probar todos los casos, fue una experiencia bastante buena gracias al aportar en el código y principalmente en la solución de ciertos casos como la documentación.

Santiago López Trujillo

Es un proyecto que fue complicado ya que depende de varias validaciones, sin embargo me gustó mucho enfrentarme a tantos retos y explorar más a fondo todos los diccionario, considero que son demasiado importantes ya que es una manera de almacenar la información de forma muy sencilla y poder obtenerla en cualquier lugar.



REFERENCIAS

Harendra. (2026, 18 de enero). *Algoritmo de retroceso*. Geeksforgeeks.
<https://www.geeksforgeeks.org/dsa/backtracking-algorithms/>

Jalswal, S. (2024, 24 de febrero). *Tutorial de comprensión del diccionario Python*. Datacamp. <https://www.datacamp.com/es/tutorial/python-dictionary-comprehension>

OMES. (2023, 20 de noviembre). *Diccionarios | Curso de Python desde cero*
 Youtube. <https://www.youtube.com/watch?v=js9NsRzjxkM>